SQL to MongoDB Mapping Chart

In addition to the charts that follow, you might want to consider the *Frequently Asked Questions* section for a selection of common questions about MongoDB.

Terminology and Concepts

The following table presents the various SQL terminology and concepts and the corresponding MongoDB terminology and concepts.

SQL Terms/Concepts	MongoDB Terms/Concepts
database	*database*
table	*collection*
row	*document* or *BSON* document
column	*field*
index	*index*
table joins	embedded documents and linking
primary key Specify any unique column or column combination as primary key.	*primary key* In MongoDB, the primary key is automatically set to the *_id* field.
aggregation (e.g. group by)	aggregation pipeline See the *SQL to Aggregation Mapping Chart*.

Executables

The following table presents some database executables and the corresponding MongoDB executables. This table is *not* meant to be exhaustive.

	MongoDB	MySQL	Oracle	Informix	DB2
Database Server	mongod	mysqld	oracle	IDS	DB2 Server
Database Client	mongo	mysql	sqlplus	DB-Access	DB2 Client

Examples

The following table presents the various SQL statements and the corresponding MongoDB statements. The examples in the table assume the following conditions:

- The SQL examples assume a table named users.
- The MongoDB examples assume a collection named users that contain documents of the following prototype:
- `{`
- `_id: ObjectId("509a8fb2f3f4948bd2f983a0"),`
- `user_id: "abc123",`
- `age: 55,`
- `status: 'A'`
- `}`

Create and Alter

The following table presents the various SQL statements related to table-level actions and the corresponding MongoDB statements.

SQL Schema Statements	MongoDB Schema Statements

`CREATE TABLE users (`
`  id MEDIUMINT NOT NULL`
`  AUTO_INCREMENT,`
`  user_id Varchar(30),`
`  age Number,`
`  status char(1),`
`  PRIMARY KEY (id)`
`)`
	Implicitly created on first insert() operation. The primary key _id is automatically added if _id field is not specified.
	
	`db.users.insert( {`
	`  user_id: "abc123",`
	`  age: 55,`
	`  status: "A"`
	` } )`
	
	However, you can also explicitly create a collection:
	
	`db.createCollection("users")`

`ALTER TABLE users`
`ADD join_date DATETIME`
	Collections do not describe or enforce the structure of its documents; i.e. there is no structural alteration at the collection level.
	
	However, at the document level, update() operations can add fields to existing documents using the $set operator.
	
	`db.users.update(`
	`  { },`
	`  { $set: { join_date: new Date() } },`
	`  { multi: true }`
	`)`

SQL Schema Statements	MongoDB Schema Statements

`ALTER TABLE users`
`DROP COLUMN join_date`
	Collections do not describe or enforce the structure of its documents; i.e. there is no structural alteration at the collection level.
	
	However, at the document level, update() operations can remove fields from documents using the $unset operator.
	
	`db.users.update(`
	`  { },`
	`  { $unset: { join_date: "" } },`
	`  { multi: true }`
	`)`

`CREATE INDEX`
`idx_user_id_asc`
`ON users(user_id)`
	`db.users.createIndex( { user_id: 1 } )`

`CREATE INDEX`
 	
`idx_user_id_asc_age_desc`
`ON users(user_id, age`
`DESC)`
	`db.users.createIndex( { user_id: 1, age: -1 } )`

`DROP TABLE users`	`db.users.drop()`

For more information, see db.collection.insert(), db.createCollection(), db.collection.update(), $set, $unset, db.collection.createIndex(), *indexes*, db.collection.drop(), and *Data Modeling Concepts*.

Insert

The following table presents the various SQL statements related to inserting records into tables and the corresponding MongoDB statements.

SQL INSERT Statements	MongoDB insert() Statements

`INSERT INTO users(user_id,`
`  age,`
`  status)`
`VALUES ("bcd001",`
`  45,`
`  "A")`
	`db.users.insert(`
	`  { user_id: "bcd001", age: 45, status: "A" }`
	`)`

For more information, see db.collection.insert().

Select

The following table presents the various SQL statements related to reading records from tables and the corresponding MongoDB statements.

SQL SELECT Statements	MongoDB find() Statements
SELECT * FROM users	db.users.find()
SELECT id, user_id, status FROM users	db.users.find({ }, { user_id: 1, status: 1 })
SELECT user_id, status FROM users	db.users.find({ }, { user_id: 1, status: 1, _id: 0 })
SELECT * FROM users WHERE status = "A"	db.users.find({ status: "A" })
SELECT user_id, status FROM users WHERE status = "A"	db.users.find({ status: "A" }, { user_id: 1, status: 1, _id: 0 })
SELECT * FROM users WHERE status != "A"	db.users.find({ status: { $ne: "A" } })
SELECT * FROM users WHERE status = "A" AND age = 50	db.users.find({ status: "A", age: 50 })
SELECT * FROM users WHERE status = "A" OR age = 50	db.users.find({ $or: [{ status: "A" } , { age: 50 }] })
SELECT * FROM users WHERE age > 25	db.users.find({ age: { $gt: 25 } })
SELECT * FROM users WHERE age < 25	db.users.find({ age: { $lt: 25 } })
SELECT * FROM users WHERE age > 25 AND age <= 50	db.users.find({ age: { $gt: 25, $lte: 50 } })
SELECT * FROM users WHERE user_id like "%bc%"	db.users.find({ user_id: /bc/ })
SELECT * FROM users WHERE user_id like "bc%"	db.users.find({ user_id: /^bc/ })
SELECT * FROM users	db.users.find({ status: "A" }).sort({ user_id: 1 })

SQL SELECT Statements	MongoDB find() Statements
WHERE status = "A" ORDER BY user_id ASC	
SELECT * FROM users WHERE status = "A" ORDER BY user_id DESC	db.users.find({ status: "A" }).sort({ user_id: -1 })
SELECT COUNT(*) FROM users	db.users.count() *or* db.users.find().count()
SELECT COUNT(user_id) FROM users	db.users.count({ user_id: { $exists: true } }) *or* db.users.find({ user_id: { $exists: true } }).count()
SELECT COUNT(*) FROM users WHERE age > 30	db.users.count({ age: { $gt: 30 } }) *or* db.users.find({ age: { $gt: 30 } }).count()
SELECT DISTINCT(status) FROM users	db.users.distinct("status")
SELECT * FROM users LIMIT 1	db.users.findOne() *or* db.users.find().limit(1)
SELECT * FROM users LIMIT 5 SKIP 10	db.users.find().limit(5).skip(10)
EXPLAIN SELECT * FROM users WHERE status = "A"	db.users.find({ status: "A" }).explain()

For more information, see db.collection.find(), db.collection.distinct(), db.collection.findOne(), $ne $and, $or, $gt, $lt, $exists, $lte, $regex, limit(), skip(), explain(), sort(), and count().

Update Records

The following table presents the various SQL statements related to updating existing records in tables and the corresponding MongoDB statements.

SQL Update Statements	MongoDB update() Statements

`UPDATE users`
`SET status = "C"`
`WHERE age > 25`
	`db.users.update(`
	`  { age: { $gt: 25 } },`
	`  { $set: { status: "C" } },`
	`  { multi: true }`
	`)`

`UPDATE users`
`SET age = age + 3`
`WHERE status = "A"`
	`db.users.update(`
	`  { status: "A" } ,`
	`  { $inc: { age: 3 } },`
	`  { multi: true }`
	`)`

For more information, see db.collection.update(), $set, $inc, and $gt.

Delete Records

The following table presents the various SQL statements related to deleting records from tables and the corresponding MongoDB statements.

SQL Delete Statements	MongoDB remove() Statements
`DELETE FROM users` `WHERE status = "D"`	`db.users.remove( { status: "D" } )`
`DELETE FROM users`	`db.users.remove({})`

For more information, see db.collection.remove().

SQL to Aggregation Mapping Chart

The *aggregation pipeline* allows MongoDB to provide native aggregation capabilities that corresponds to many common data aggregation operations in SQL.

The following table provides an overview of common SQL aggregation terms, functions, and concepts and the corresponding MongoDB *aggregation operators*:

SQL Terms, Functions, and Concepts	MongoDB Aggregation Operators
WHERE	$match
GROUP BY	$group
HAVING	$match
SELECT	$project
ORDER BY	$sort
LIMIT	$limit
SUM()	$sum
COUNT()	$sum
join	No direct corresponding operator; *however*, the $unwind operator allows for somewhat similar functionality, but with fields embedded within the document.

Examples

The following table presents a quick reference of SQL aggregation statements and the corresponding MongoDB statements. The examples in the table assume the following conditions:

- The SQL examples assume *two* tables, orders and order_lineitem that join by the order_lineitem.order_id and the orders.id columns.
- The MongoDB examples assume *one* collection orders that contain documents of the following prototype:

```
{
  cust_id: "abc123",
  ord_date: ISODate("2012-11-02T17:04:11.102Z"),
  status: 'A',
  price: 50,
  items: [ { sku: "xxx", qty: 25, price: 1 },
           { sku: "yyy", qty: 25, price: 1 } ]
}
```

SQL Example	MongoDB Example	Description

SELECT COUNT(*)
AS count
FROM orders
	db.orders.aggregate([
	{
	$group: {
	_id: null,
	count: { $sum: 1
	}
	}
	}
	])
		Count all records from orders

SELECT
SUM(price) AS
total
FROM orders
	db.orders.aggregate([
	{
	$group: {
	_id: null,
	total: { $sum:
	"$price" }
	}
	}
	])
		Sum the price field from orders

SELECT cust_id,
 	
SUM(price) AS
total
FROM orders
GROUP BY cust_id
	db.orders.aggregate([
	{
	$group: {
	_id: "$cust_id",
	total: { $sum:
	"$price" }
	}
	}
	])
		For each unique cust_id, sum the price field.

SELECT cust_id,
 	
SUM(price) AS
total
FROM orders
GROUP BY cust_id
ORDER BY total
	db.orders.aggregate([
	{
	$group: {
	_id: "$cust_id",
	total: { $sum:
	"$price" }
	}
	},
	{ $sort: { total: 1 }
	}
	])
		For each unique cust_id, sum the price field, results sorted by sum.

SELECT cust_id,
ord_date,
 	
SUM(price) AS
total
FROM orders
GROUP BY
cust_id,
 	
ord_date
	db.orders.aggregate([
	{
	$group: {
	_id: {
	cust_id:
	"$cust_id",
	ord_date: {
	month: {
	$month: "$ord_date" },
	day: {
	$dayOfMonth: "$ord_date"
	},
	year: {
	$year: "$ord_date"}
		For each unique cust_id, ord_date grouping, sum the price field. Excludes the time portion of the date.

SQL Example	MongoDB Example	Description

	`}`
	`},`
	`total: { $sum: "$price" }`
	`}`
	`}`
	`] )`	

`SELECT cust_id,`
`count(*)`
`FROM orders`
`GROUP BY cust_id`
`HAVING count(*) > 1`
	`db.orders.aggregate( [`
	`{`
	`$group: {`
	`_id: "$cust_id",`
	`count: { $sum: 1 }`
	`}`
	`},`
	`{ $match: { count: { $gt: 1 } } }`
	`] )`
		For cust_id with multiple records, return the cust_id and the corresponding record count.

`SELECT cust_id,`
`ord_date,`
 	
`SUM(price) AS total`
`FROM orders`
`GROUP BY cust_id,`
 	
`ord_date`
`HAVING total > 250`
	`db.orders.aggregate( [`
	`{`
	`$group: {`
	`_id: {`
	`cust_id: "$cust_id",`
	`ord_date: {`
	`month: { $month: "$ord_date" },`
	`day: { $dayOfMonth: "$ord_date" },`
	`year: { $year: "$ord_date"}`
	`}`
	`},`
	`total: { $sum: "$price" }`
	`}`
	`},`
	`{ $match: { total: { $gt: 250 } } }`
	`] )`
		For each unique cust_id, ord_date grouping, sum the price field and return only where the sum is greater than 250. Excludes the time portion of the date.

`SELECT cust_id,`
 	
`SUM(price) as total`
`FROM orders`
`WHERE status = 'A'`
`GROUP BY cust_id`
	`db.orders.aggregate( [`
	`{ $match: { status: 'A' } },`
	`{`
	`$group: {`
	`_id: "$cust_id",`
	`total: { $sum: "$price" }`
	`}`
	`}`
	`] )`
		For each unique cust_id with status A, sum the price field.

SQL Example	MongoDB Example	Description

SELECT cust_id,
 	
SUM(price) as total
FROM orders
WHERE status = 'A'
GROUP BY cust_id
HAVING total > 250
	db.orders.aggregate([
	  { $match: { status: 'A' } },
	  {
	  $group: {
	  _id: "$cust_id",
	  total: { $sum: "$price" }
	  }
	  },
	  { $match: { total: { $gt: 250 } } }
	])
		For each unique cust_id with status A, sum the price field and return only where the sum is greater than 250.

SELECT cust_id,
 	
SUM(li.qty) as qty
FROM orders o,
 	
order_lineitem li
WHERE li.order_id = o.id
GROUP BY cust_id
	db.orders.aggregate([
	  { $unwind: "$items" },
	  {
	  $group: {
	  _id: "$cust_id",
	  qty: { $sum: "$items.qty" }
	  }
	  }
	])
		For each unique cust_id, sum the corresponding line item qty fields associated with the orders.

SELECT COUNT(*)
FROM (SELECT cust_id,
 	
ord_date
  FROM orders
  GROUP BY cust_id,
 	
ord_date)
  as DerivedTable
	db.orders.aggregate([
	  {
	  $group: {
	  _id: {
	  cust_id: "$cust_id",
	  ord_date: {
	  month: { $month: "$ord_date" },
	  day: { $dayOfMonth: "$ord_date" },
	  year: { $year: "$ord_date"}
	  }
	  }
	  }
	  },
	  {
	  $group: {
	  _id: null,
	  count: { $sum: 1 }
	  }
	  }
	])	